

ProcRosetta

Aligning Event Logs, Process Trees, and Petri Nets
in a Shared Latent Space with Grammar-Constrained Decoding

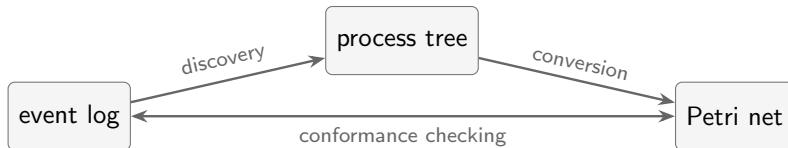
Alessandro Berti Wil M.P. van der Aalst

Process and Data Science (PADS), RWTH Aachen University

ICPM 2027

The Problem

- Process mining uses **three views** of the same behavior: **event logs**, **process trees**, **Petri nets**
- Every bridge between them is a **dedicated algorithm**: discovery, conformance checking, model transformation
- Missing: a **common representation** in which the three views of one process are **the same point**



The Goal

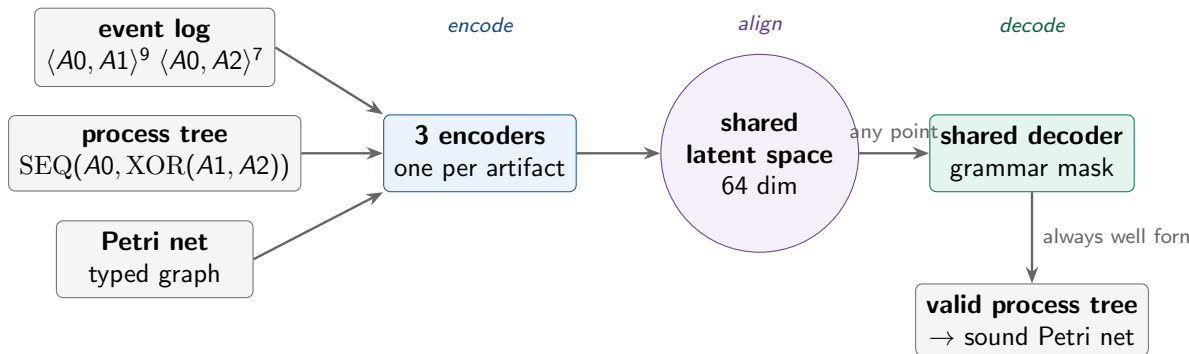
One latent space of process behavior

Embed **logs**, **trees**, and **nets** so that three views of the **same process** land on **nearby points**, and **decode** any point back into a **valid process model**.

- **RQ1**: do the three views of one process receive nearby embeddings?
- **RQ2**: can logs and nets be translated into **valid** process trees?
- **RQ3**: how does this compare with classical features and the Inductive Miner?

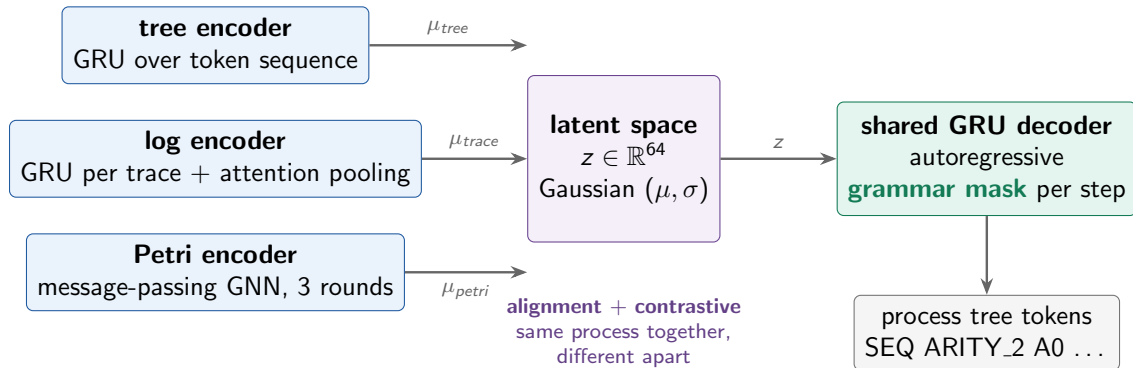
The name: like the **Rosetta stone**, the training data is the same content in three languages.

The Approach at a Glance



- Same process $\Rightarrow$ views **pulled together**; different processes **pushed apart**
- Decoding realizes **discovery** (log $\rightarrow$ tree), **translation** (net $\rightarrow$ tree), **auto-encoding**

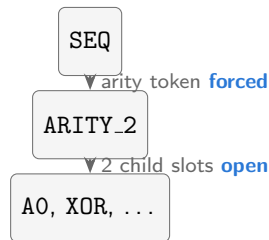
Architecture



- Deliberately **small**: about **0.5M parameters**, trains in 30 min on CPU

Grammar Masking: Validity by Construction

- The decoder tracks **open node slots** and pending arities
- At every step, **grammar-invalid tokens** get probability **zero**
- Every terminated output **parses** into a process tree
- Deterministic conversion yields a **sound Petri net**



<eos> allowed **only when**
no slot is open

100% of 2,048 test decodes: valid tree, convertible to a Petri net

Synthetic paired triples (correspondence by construction):

- 1 **sample** a random process tree
(SEQ, XOR, AND; depth ≤ 3 ; ≤ 30 activities)
- 2 **simulate** it into an event log
(16 traces per sample)
- 3 **convert** it into a Petri net
(deterministic, via PM4Py)

Split	Triples
Training	4,000
Validation	512
Test	512

Fixed seed, disjoint splits.
Avg. tree size ≈ 10 nodes.

Training Loss

All three artifacts of a triple decode to the **same target tree**:

$$\mathcal{L} = \underbrace{\mathcal{L}_{tree \rightarrow tree} + \mathcal{L}_{log \rightarrow tree} + \mathcal{L}_{petri \rightarrow tree}}_{\text{reconstruction and translation}} + 0.1 \underbrace{\mathcal{L}_{align}}_{\text{pull together}} + 0.1 \underbrace{\mathcal{L}_{contrastive}}_{\text{push apart}} + 0.001 \mathcal{L}_{KL}$$

- **Decode losses**: grammar-masked cross-entropy (teacher forcing) for reconstruction (tree $\rightarrow$ tree) and the two translations (log $\rightarrow$ tree, Petri net $\rightarrow$ tree)
- **Alignment**: squared distance between the latent means of the three views: **same process, same point**
- **Contrastive** (CLIP style): each artifact must be most similar to its **own counterpart** in the batch: **different processes apart**
- **KL**: weak pull toward a standard normal keeps the space **bounded and smooth**

All on the **held-out test split** (512 triples), best-validation checkpoint:

- **Decode quality**: validity, exact match, edit distance, **behavioral distance** of decoded model vs. source log
- **Discovery quality**: log $\rightarrow$ model vs. the **Inductive Miner**, scored by alignment-based fitness and precision
- **Embedding quality**: does latent distance correlate with **behavioral distance**? vs. 7 baselines incl. PetriNet2Vec
- **Cross-modal retrieval**: given a net, find its log among 512 candidates

Evaluation: Results

- **100%** valid, Petri-convertible decodes from all modalities
- Discovery: F1 **0.787** vs. **0.857** Inductive Miner
(92% of IM, on IM's home turf)
- Embedding quality via ρ = **Spearman rank correlation**: do **behaviorally close** logs get **close embeddings**?
- Retrieval top-1: **14 to 44%** vs. **0.2%** chance

Decoded models: behaviorally **twice as close** to their source as unrelated ones; the three encoders converge to **one geometry**.

Decode from	Valid	Exact
tree	100%	48%
log	100%	40%
net	100%	36%

Log representation	Dim.	ρ
ProcRosetta (learned)	64	0.630
directly-follows relations	332	0.736
trace-variant distribution	2,548	0.614

A compact **64-dim** learned vector rivals feature vectors **5 to 40× larger** that partly **define** the target distance.

What Can Be Improved: Data

- **Only synthetic** data: all triples come from small random block-structured trees, so results hold **in distribution** only; training and validating on **real-life logs** and model collections would test whether the space transfers to realistic behavior
- **Clean logs**: every log is a perfect, noise-free playout of its tree; real logs contain **noise, incomplete cases, deviations, long-tail variants**, and a model trained on them would have to **abstract** instead of just compress
- **Possible near-duplicates** across splits: splits are random, so a test behavior can resemble a training behavior and **inflate exact-match rates**; explicit **behavioral deduplication** would make the numbers stricter
- **Single seed**: one data configuration, one trained model; **multi-seed runs** would quantify how stable the results are

What Can Be Improved: Architecture

- **Small GRU-based encoders and decoder** (about 0.5M parameters): **scaling** them, or replacing GRUs with **transformers**, should close part of the gap to the Inductive Miner, especially on longer trees and larger logs
- **Block-structured output only**: the decoder can express only what a process tree can express; a richer **sound** target language such as **POWL** (partially ordered workflow models) needs just a new **prefix grammar** and deterministic Petri-net conversion, the rest of the framework stays unchanged
- **Only three modalities**: the latent space is open; attaching **new encoders**, e.g. for **natural-language** process descriptions or BPMN, would let text, logs, and models meet in **one space**, closer to a true Rosetta stone

One latent space, three artifact types, **valid models by construction**.

- Three encoders converge to **one geometry** of process behavior (RQ1)
- Grammar masking: **100%** valid, sound outputs (RQ2)
- Learned translation reaches **92%** of the Inductive Miner F1; 64-dim embeddings rival much larger deterministic features (RQ3)

Code, data generator, and evaluation suite:
<https://github.com/fit-alessandro-berti/proc-rosetta>

Appendix

Detailed Evaluation Results

All numbers on the held-out test split (512 triples).

Appendix: Decode Quality

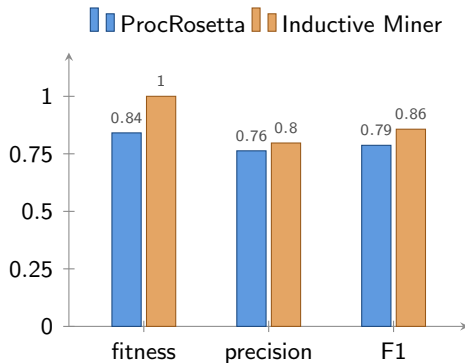
Greedy decoding from each latent source (512 samples each):

Latent source	Valid tree	Exact tree	Norm. edit ↓	Behavior L_1 ↓
μ_{tree} (tree encoder)	100%	48.4%	0.178	0.675
μ_{trace} (log encoder)	100%	40.0%	0.287	0.755
μ_{petri} (Petri encoder)	100%	35.9%	0.282	0.839
μ_{fused} (mean of three)	100%	44.3%	0.230	0.713

- **Behavior L_1** : distance between the original log and traces simulated from the decoded model, range $[0, 2]$; two **unrelated** test logs average **1.566**
- Ordering is intuitive: the tree embedding keeps the most information, then the log, then the **label-free** Petri net structure

Appendix: Discovery Quality vs. Inductive Miner

- Each of the 512 test logs: encode, decode a tree, convert to a Petri net; scored by **alignment-based** fitness and precision
- Test logs are noise-free playouts of block-structured trees: the Inductive Miner's **best case** (rediscovery guarantee, fitness 1.0)
- ProcRosetta compresses the whole log into **one 64-dim vector**, no search or replay, and still reaches **92%** of the IM's F1



Appendix: Embedding Quality

Spearman ρ vs. behavioral distance over all 130,816 log pairs; NN = mean behavioral distance to the nearest embedding neighbor (random pair: 1.566):

Method	Type	Dim.	$\rho \uparrow$	NN $\downarrow$
Directly-follows distribution [†]	log features	332	0.736	0.784
PetriNet2Vec	learned (net)	64	0.643	1.115
Activity counts	log features	20	0.635	0.946
ProcRosetta μ_{trace}	learned (log)	64	0.630	0.794
ProcRosetta μ_{fused}	learned (all)	64	0.628	0.804
Trace-variant distribution [†]	log features	2,548	0.614	0.781
ProcRosetta μ_{tree}	learned (tree)	64	0.611	0.836
ProcRosetta μ_{petri}	learned (net)	64	0.574	0.913
Eventually-follows distribution	log features	347	0.412	0.881

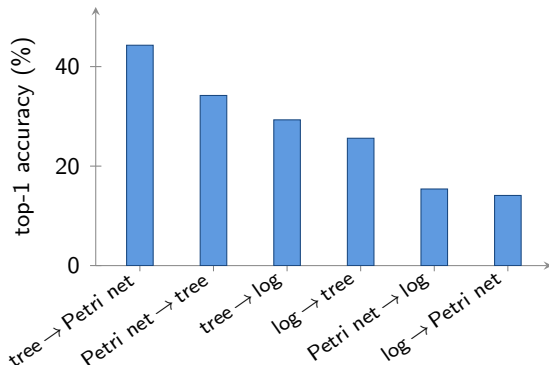
[†] shares components with the behavioral-distance definition: a **skyline**, not a fair competitor. Among the rest, ProcRosetta's log embedding is the **strongest all-round performer** at only 64 dimensions.

Appendix: Cross-Modal Retrieval

Query → target	Top-1	MRR	Rank
tree → Petri net	44.3%	0.523	36.2
Petri net → tree	34.2%	0.442	38.0
tree → log	29.3%	0.383	32.4
log → tree	25.6%	0.354	33.2
Petri net → log	15.4%	0.236	49.4
log → Petri net	14.1%	0.236	48.2

MRR = mean reciprocal rank: the average of $1/\text{rank}$ of the true counterpart; **1.0** means it is always ranked **first**, **0.5** means it is typically **second**.

Chance level: top-1 = **0.2%**, mean rank $\approx$ **256**.



Instance-level matching: the correct counterpart among **512 candidates**, **70 to 220×** above chance.